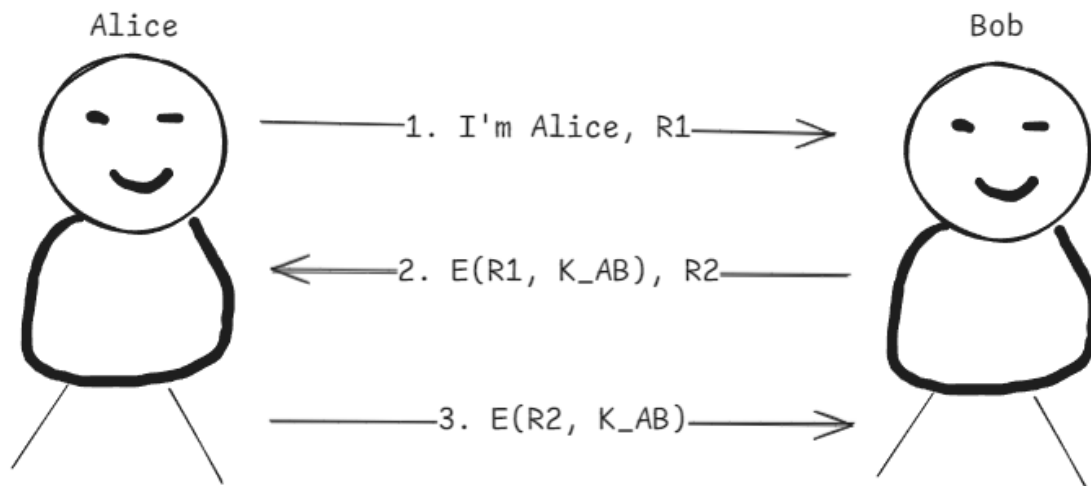


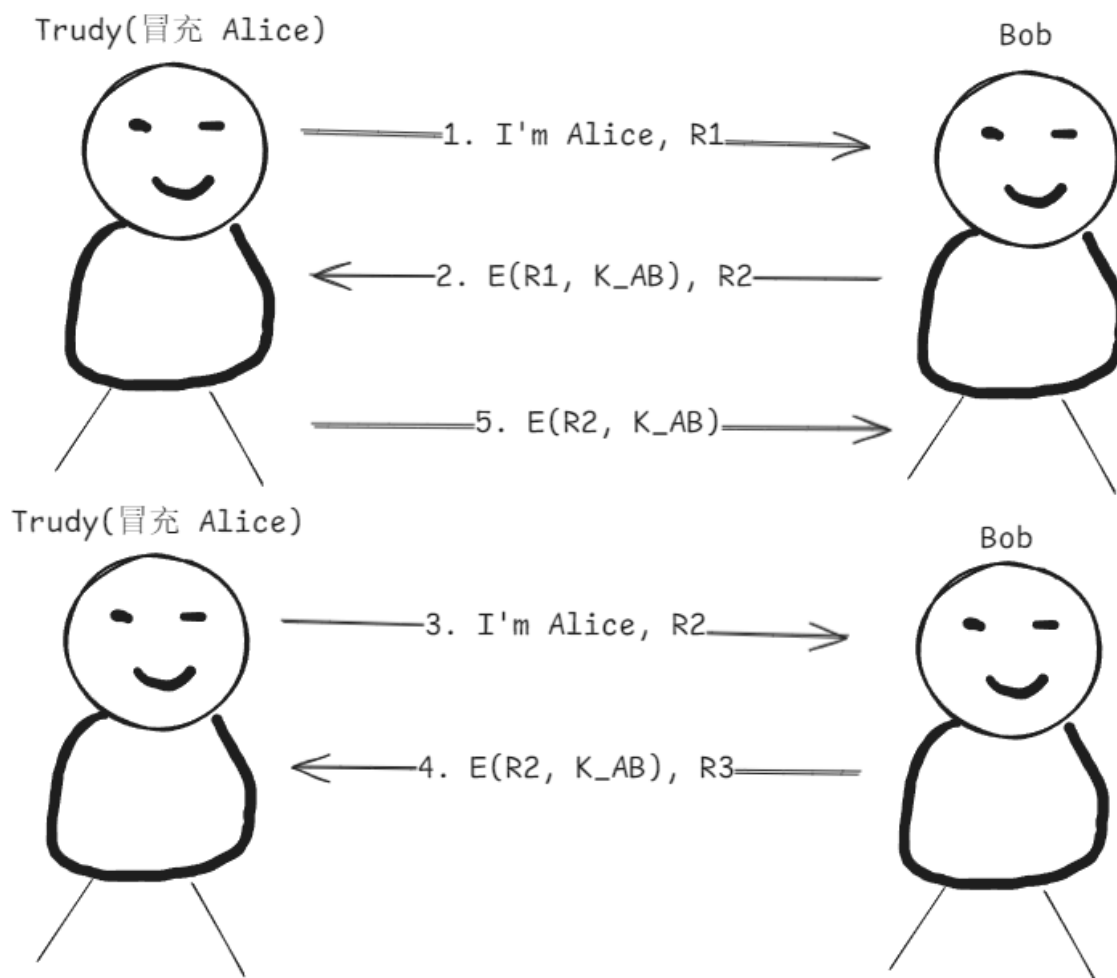
1、简单安全协议

a. 1)

Alice 和 Bob 通信流程如图所示：



Trudy 冒充 Alice 和 Bob 通信流程如图所示：



a. 2)

Trudy的反射攻击如下：

- **启动会话 1：** Trudy 冒充 Alice 向 Bob 发送身份声明及挑战值 R_1 。
- **接收挑战：** Bob 收到后，用共享密钥 K_{AB} 加密 R_1 作为响应，同时发出自己的挑战值 R_2 传给 Trudy。
- **利用会话 2 反射：** Trudy 无法计算 $E(R_2, K_{AB})$ 。为了通过 Bob 的认证，她立即向 Bob 发起一个并行会话 2，并直接把 Bob 刚才出的难题 R_2 当作自己的挑战值发送给 Bob。
- **借助 Bob 解题：** Bob 在会话 2 中并不知情，用密钥 K 加密 R_2 并将 $E(R_2, K_{AB})$ 传给 Trudy。
- **通过验证：** Trudy 利用会话 2 中由 Bob 加密的 $E(R_2, K_{AB})$ ，将其作为会话 1 的响应发给 Bob。Bob 验证无误，Trudy 成功在未掌握密钥的前提下通过了 Bob 的双向认证。

失败的根本原因：

- 协议允许攻击者将同一实体（Bob）当作免费的加密工具，去回答该实体自己在另一个会话里提出来的挑战。
- 协议没有对“谁是挑战方、谁是响应方”做出区分。

a. 3)

最小修改方案：在密文中加入身份绑定，Bob 响应 Alice 为 $E(Bob, R_1, K_{AB})$ ，Alice 响应 Bob 为 $E(Alice, R_2, K_{AB})$

能防住反射攻击的原因：当 Trudy 在会话 1 中收到 Bob 的挑战 R_2 时，Bob 预期在会话 1 中拿到的正确响应是 $E(Alice, R_2, K_{AB})$ 。如果 Trudy 故技重施，将 R_2 反射到会话 2 中发给 Bob，Bob 在会话 2 中只会返回 $E(Bob, R_2, K_{AB})$ 。Trudy 拿到 $E(Bob, R_2, K_{AB})$ 之后，由于处于加密状态，此时她无法将其转换为 $E(Alice, R_2, K_{AB})$ ，因此在会话 1 中无法提交正确的响应，攻击彻底失效。

b. 1)

安全属性：保密性。

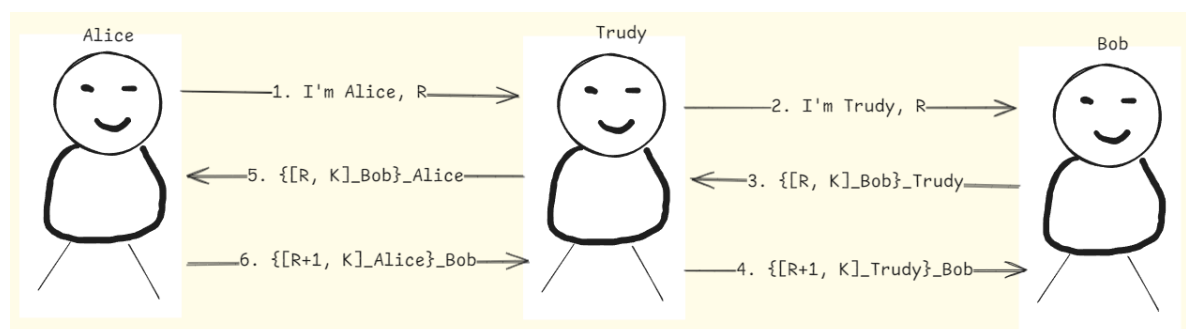
保护的关键元素：共享的会话密钥 K

b. 2)

安全属性：完整性、不可否认性、身份验证。

b. 3)

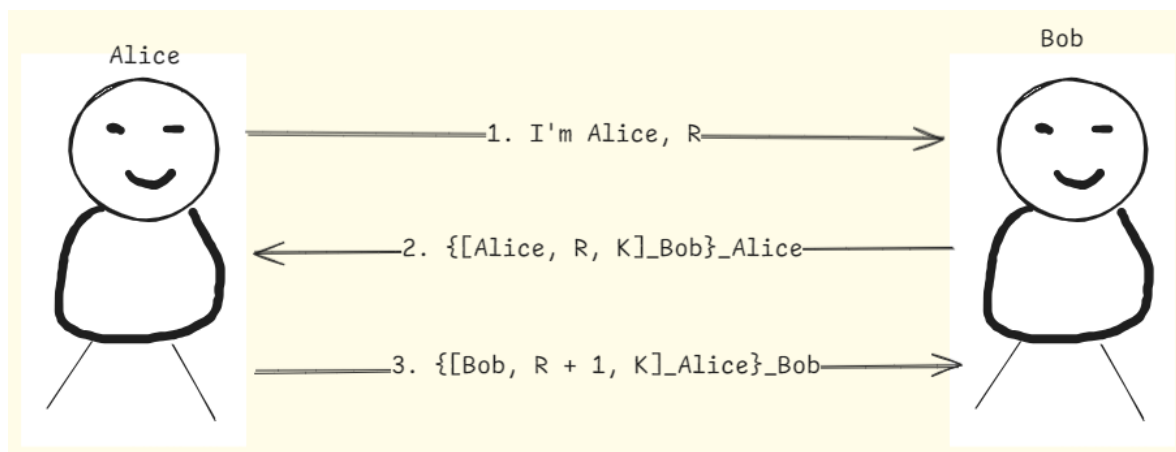
可能遭受中间人攻击，Trudy通过此攻击让 Alice 误以为和 Bob 进行通信，信任了会话密钥 K ，后续 Alice 发送的所有用 K 加密的数据，都将被 Trudy 轻易窃听和解密。攻击过程如下图所示：



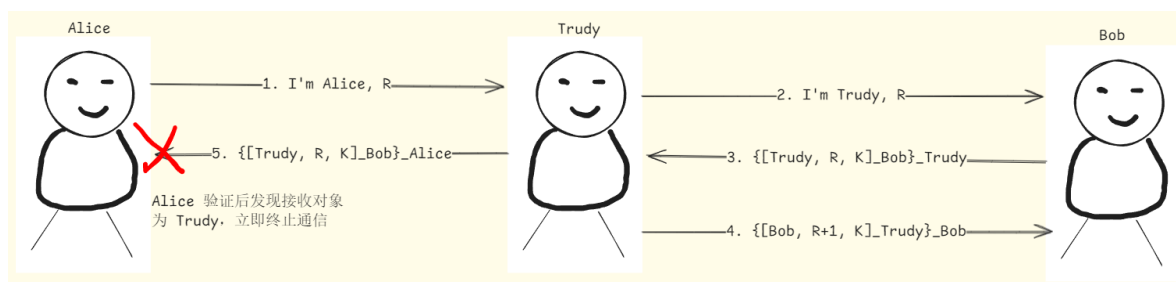
b. 4)

修改方案：必须在签名内包含希望的接收方身份

Alice 和 Bob 通信过程如下图所示：



Trudy 实施中间人攻击失败过程如下图所示：



2、现实安全协议

a.

WEP:

- 致命弱点：
 - CRC 校验的完整性问题：利用比特翻转攻击，攻击者可以在完全不知道密钥的情况下篡改密文，并对应修正 CRC 值，从而绕过完整性检查。
 - IV 问题：IV 仅有 24 位，在网络中容易发生 IV 重复（使用相同的 K，产生相同密钥流），导致攻击者可通过异或（XOR）密文恢复明文。
 - 密码分析攻击问题：通过特定的 IV 可以破解长期密钥 K 的一部分，如果使用足够的 IV，攻击者就能轻松找到长期密钥 K。
- 现代协议的修复：
 - WPA2 (CCMP) / WPA3 (CCMP / GCMP) 彻底废弃了 RC4，改用 AES 加密。
 - WPA2 完整性保护：CBC-MAC
 - WPA3 完整性保护：Galois Field 运算

GSM:

- 致命弱点：
 - 假基站：仅网络侧对移动电话（SIM 卡）进行鉴权，终端无法对网络进行身份验证。攻击者可架设假基站强行吸附终端并窃听通信。
 - 加密技术薄弱：A5/1 算法极易被已知明文破解。
 - SIM 卡问题：
 - A3 / A8 算法使用的哈希值为 COMP128，被 150,000 条选定的明文破解。使用 SIM 卡，可在 2 至 10 小时内获得 Ki 值。
 - 攻击者使用光学故障感应和分区攻击，并掌握了 SIM 卡，就能在几秒钟内恢复 Ki 值。
 - DOS 攻击：拒绝服务攻击极易实施且效果显著。

- 加密边界过窄：加密仅存在于移动电话到基站之间，基站与基站控制器之间没有加密。
- 现代协议的修复：
 - 引入了基于 AKA 协议的双向认证机制。
 - 采用现代密码算法（如 AES、SNOW 3G、ZUC）。

SSL：

- 致命弱点：
 - 握手无哈希致降级
 - MAC-then-Encrypt结构致POODLE攻击
- 现代协议的修复：
 - 全文本哈希
 - 强制AEAD

b. 1)

传统缺陷：VPN盲信内网，缺乏隔离致凭证泄露后黑客极易横向移动；Kerberos刚性依赖时钟与域控直连，不适用Web/API动态跨域场景。

b. 2)

零信任原则：“持续验证，永不信任”。网络位置与IP不再被信任；访问控制全面基于强身份、端点合规状态、环境及行为风险评分等动态多维上下文进行决策。

b. 3)

替代框架：OAuth 2.0（面向API的分布式令牌授权框架）；OIDC（构建在OAuth之上用于Web/移动端跨域SSO的身份认证协议）；SAML 2.0（基于XML断言的企业级联合单点登录标准）。

3、 软件安全

a.

- **定义与原理**：“金丝雀”（Canary）技术是用于检测栈溢出的主动防护机制。原理是在局部变量缓冲区与栈返回地址之间插入一个随机生成的安全 Cookie 值。
- **防溢出机制**：发生溢出时，攻击者篡改返回地址必然会先覆盖该金丝雀值。在函数返回前，程序会校验此 Cookie，若发现改变则判定栈已被破坏，立即中止运行，从而防止控制流被劫持。
- **微软 /GS 选项**：微软在编译器 /GS 选项中实现了此技术。启用后，编译器会在易受攻击的函数入口处注入代码将 Cookie 存入栈中，并在退出前进行验证。若不匹配则直接触发异常终止进程，保障软件安全。

b.

Trudy 完全可以利用该漏洞发动拒绝服务（DoS）攻击。

原因：NX（不可执行）位仅能限制栈、堆等数据区的执行权限，防止运行注入的恶意 Shellcode，但它无法阻止溢出发生，也无法保护返回地址不被篡改。当 Trudy 利用溢出将返回地址篡改为非法或不可访问的随机地址时，虽因 NX 限制无法运行恶意代码，但在函数返回尝试跳转时，CPU 会因读取非法地址触发内存段错误，导致程序异常崩溃。如果目标是核心系统服务，持续引发发现持崩溃就会使服务完全停摆，从而成功实现 DoS 攻击。

4、 代码实践

a. 1)

危险原因：该函数仅使用了 AES-CBC 模式进行解密，缺乏 MAC、HMAC 等完整性校验机制。

篡改后的系统后果：

- 静默接受错误数据：系统不会抛出完整性错误，而是直接解密被篡改的密文。
- 比特翻转：由于 CBC 模式的特性，篡改密文块 C_i 中的某几个字节，会导致解密出的明文块 P_i 变成乱码，但会精确翻转下一个明文块 P_{i+1} 中对应位置的比特。攻击者可借此伪造特定的明文内容（如把 `user` 改为 `admin`）。
- 潜在的 Padding Oracle 攻击：如果底层包含填充逻辑（如 PKCS#7），这种盲目的解密极易让系统通过抛出“填充错误”或“正常处理”的行为差异，沦为 Padding Oracle，导致攻击者能够逐字节逆向出所有明文。

a. 2)

重写的加密和解密函数如下所示：

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.exceptions import InvalidTag

# 生成安全的 256-bit 密钥
key = AESGCM.generate_key(bit_length=256)

def secure_encrypt(plaintext_bytes: bytes) -> bytes:
    """
    使用 AES-GCM 模式加密
    """
    # GCM 模式推荐使用 96-bit (12 bytes) 的随机 Nonce (IV)
    nonce = os.urandom(12)
    aesgcm = AESGCM(key)

    # encrypt 会自动在密文末尾附加 16 bytes 的 Authentication Tag (MAC)
    ciphertext = aesgcm.encrypt(nonce, plaintext_bytes, associated_data=None)

    # 将 nonce 拼接到密文头部以便解密方使用
    return nonce + ciphertext

def secure_decrypt(encrypted_data: bytes) -> bytes:
    """
    使用 AES-GCM 模式解密并进行完整性验证
    """
    # 提取前 12 字节的 nonce 和后续的密文及 Tag
    nonce = encrypted_data[:12]
    ciphertext = encrypted_data[12:]
    aesgcm = AESGCM(key)

    try:
        # decrypt 会自动计算并校验密文末尾的 Tag
        plaintext = aesgcm.decrypt(nonce, ciphertext, associated_data=None)
        return plaintext
    except InvalidTag:
        # 捕获篡改行为：一旦密文、Nonce 或 Tag 被改动哪怕 1 个 bit，校验都会失败
        print("严重警告：检测到密文已被非法篡改或损坏，拒绝解密！")
        raise
```

测试及测试结果如下所示：

```
# 1. 准备原始数据
original_message = "这是需要保护的机密信息 (Top Secret Content)"
plaintext = original_message.encode('utf-8')
print(f"原始明文: {original_message}")

# 2. 执行加密
encrypted_packet = secure_encrypt(plaintext)
print(f"加密后的数据 (Nonce + Ciphertext + Tag) 长度: {len(encrypted_packet)} bytes")

# 3. 正常解密验证
decrypted_bytes = secure_decrypt(encrypted_packet)
print(f"正常解密结果: {decrypted_bytes.decode('utf-8')}")

# 4. 模拟篡改验证 (Tamper Test)
print("\n--- 正在进行篡改测试 ---")
tampered_packet = bytearray(encrypted_packet)
# 故意修改密文中的最后一个字节 (通常是 Tag 的一部分)
tampered_packet[-1] = tampered_packet[-1] ^ 0xFF

try:
    secure_decrypt(bytes(tampered_packet))
except InvalidTag:
    print("验证成功: 系统成功识别了数据篡改并拒绝了解密。")
```

原始明文: 这是需要保护的机密信息 (Top Secret Content)
加密后的数据 (Nonce + Ciphertext + Tag) 长度: 82 bytes
正常解密结果: 这是需要保护的机密信息 (Top Secret Content)

--- 正在进行篡改测试 ---
严重警告: 检测到密文已被非法篡改或损坏, 拒绝解密!
验证成功: 系统成功识别了数据篡改并拒绝了解密。

b. 1)

Payload 如下:

```
admin' OR '1'='1
```

SQL 注入原理: 当输入上述 Payload 后, 原始代码拼接成的 SQL 语句会变成: `SELECT * FROM users WHERE username='admin' OR '1'='1' AND password_hash='...'`, 由于 or 的短路求值特性, 数据库最终执行的查询仅为验证用户名是否为 `admin`, 从而完美绕过密码验证机制, 直接登录成功。

测试及测试结果如下:

```
# 1. 初始化数据库
conn = init_db()

print("---- 尝试攻击 SQL 注入漏洞 ----")

# 2. 构造攻击载荷 (Payload)
# 原理: 利用单引号闭合掉 SQL 语句中的 username 字段, 并注入 OR '1'='1'
```

```
# 最终生成的 SQL 语句类似于: SELECT * FROM users WHERE username='admin' OR '1'='1'
AND password_hash='...'
# 由于 OR 的优先级, 只要前半部分成立即可登录
attack_username = "admin' OR '1'='1"
attack_password = "wrong_password" # 密码已经不再重要

# 3. 执行测试
print(f"尝试使用用户名: {attack_username}")
print(f"尝试使用密码: {attack_password}")

is_logged_in = insecure_login(conn, attack_username, attack_password)

if is_logged_in:
    print("注入测试结果: 登录成功! 漏洞验证成功。")
else:
    print("注入测试结果: 登录失败。")
```

```
--- 尝试攻击 SQL 注入漏洞 ---
尝试使用用户名: admin' OR '1'='1
尝试使用密码: wrong_password
登录成功! 欢迎回来, admin。
注入测试结果: 登录成功! 漏洞验证成功。
```

b. 2)

重新修改后的函数代码如下:

```
import sqlite3
import hashlib

def secure_login(conn, username, password):
    pwd_hash = hashlib.md5(password.encode()).hexdigest()

    # 修复核心: 使用 ? 作为安全的占位符
    query = "SELECT * FROM users WHERE username = ? AND password_hash = ?"

    cursor = conn.cursor()
    # 将不受信任的用户输入放入元组中, 交由数据库驱动进行安全转义和绑定
    cursor.execute(query, (username, pwd_hash))
    user = cursor.fetchone()

    if user:
        print(f"登录成功! 欢迎回来, {user[1]}。")
        return True
    else:
        print("登录失败: 用户名或密码错误。")
        return False
```

测试及测试结果如下:

```
print("--- 验证修复后的安全性（参数化查询） ---")

# 使用之前相同的攻击载荷
attack_payload = "admin' OR '1'='1"
print(f"尝试注入用户名: {attack_payload}")

# 使用 secure_login 进行测试
is_safe = secure_login(conn, attack_payload, "wrong_password")

if not is_safe:
    print("验证成功: SQL 注入攻击已被拦截, 登录失败。")
else:
    print("警告: 攻击仍然成功, 请检查修复逻辑。")
```

```
--- 验证修复后的安全性（参数化查询） ---
尝试注入用户名: admin' OR '1'='1
登录失败: 用户名或密码错误。
验证成功: SQL 注入攻击已被拦截, 登录失败。
```